Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

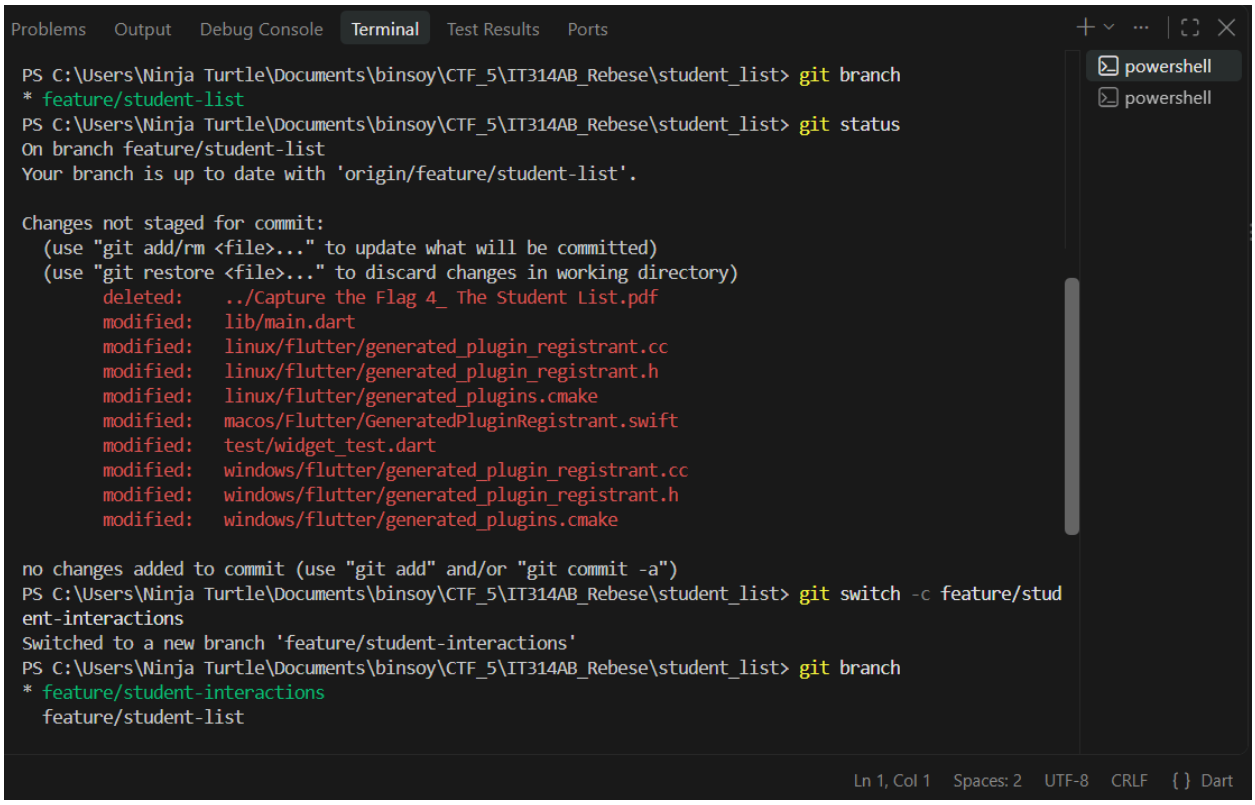
Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

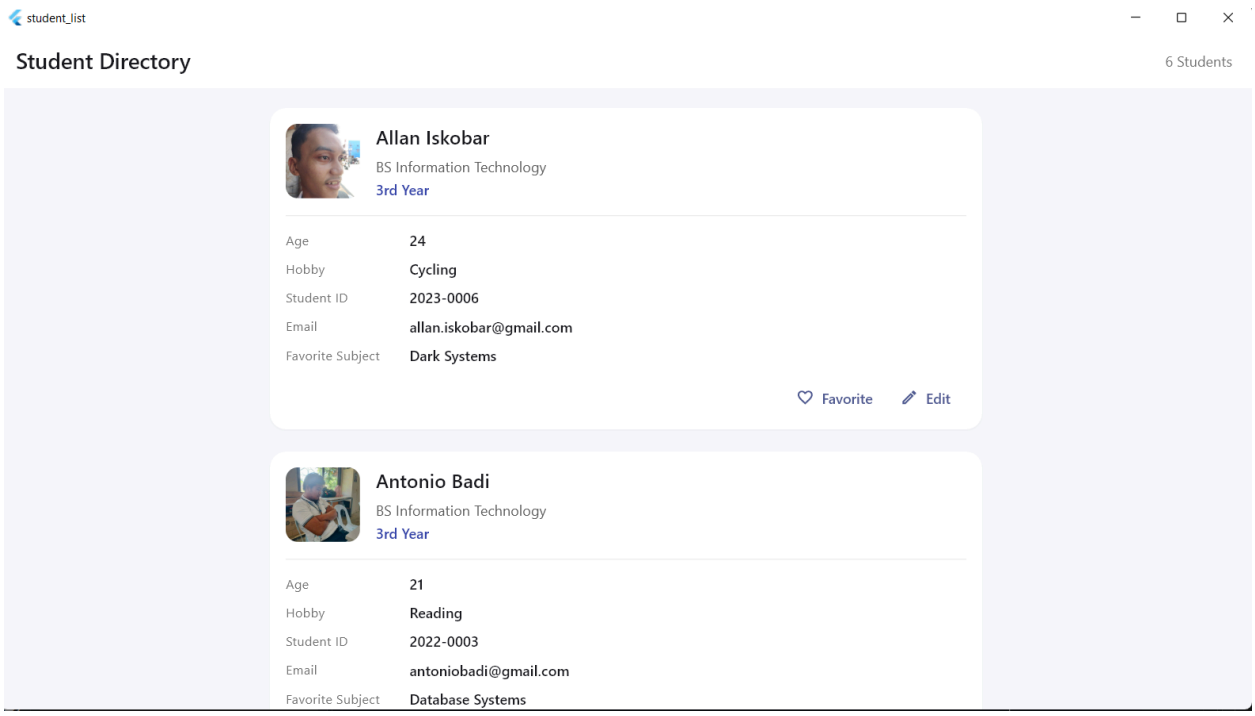
The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

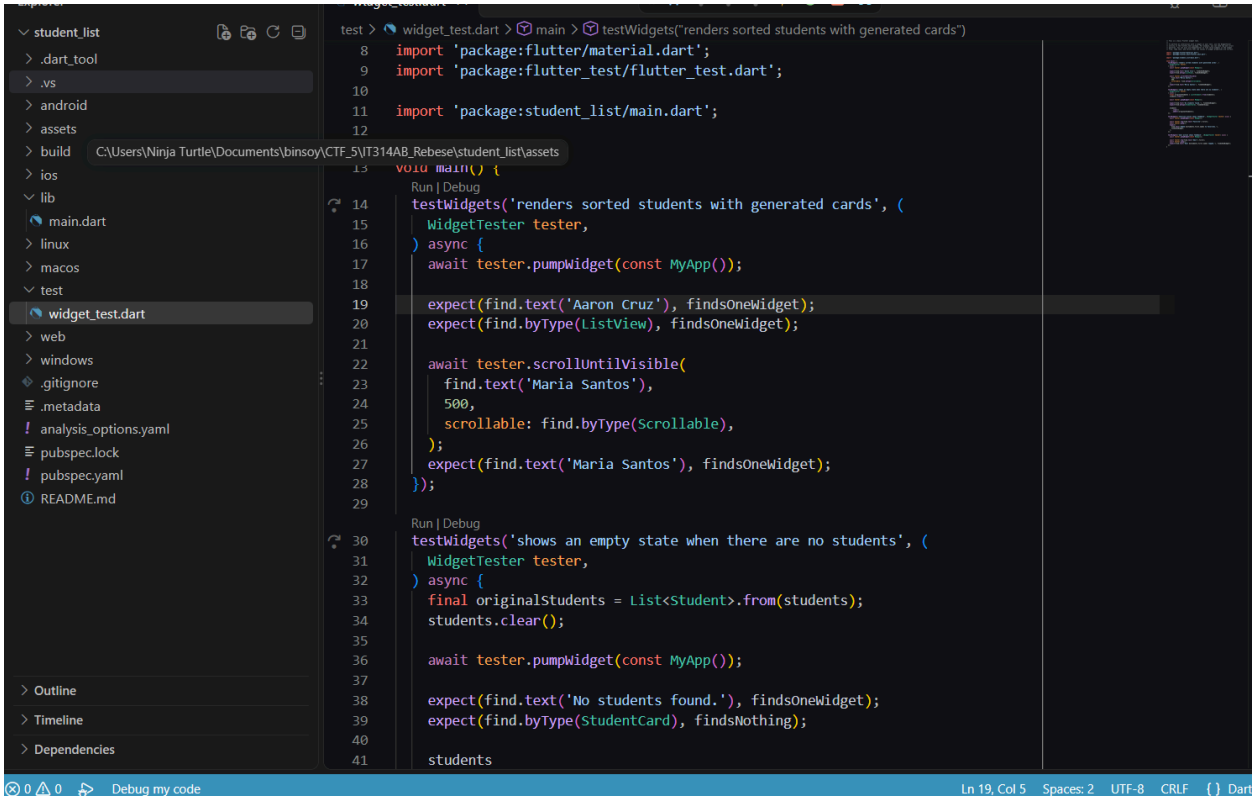
When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

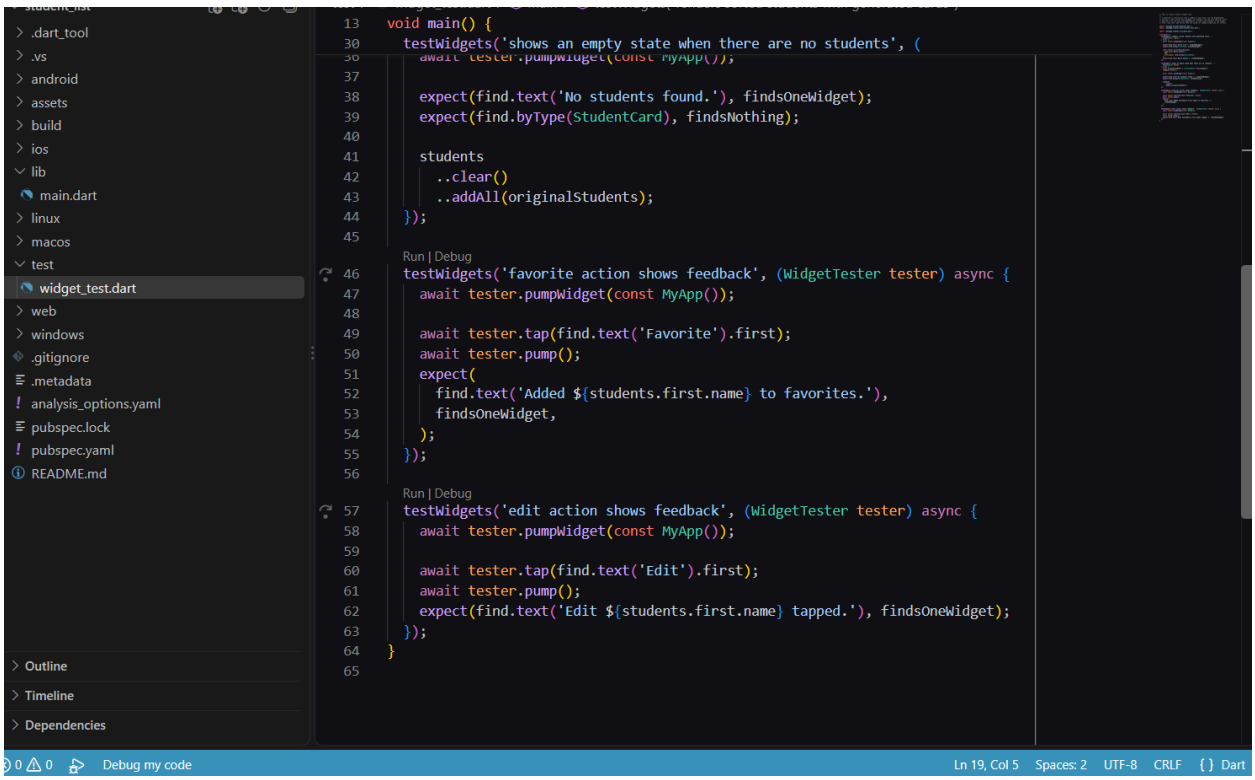
Screenshot:

- Student Card with buttons

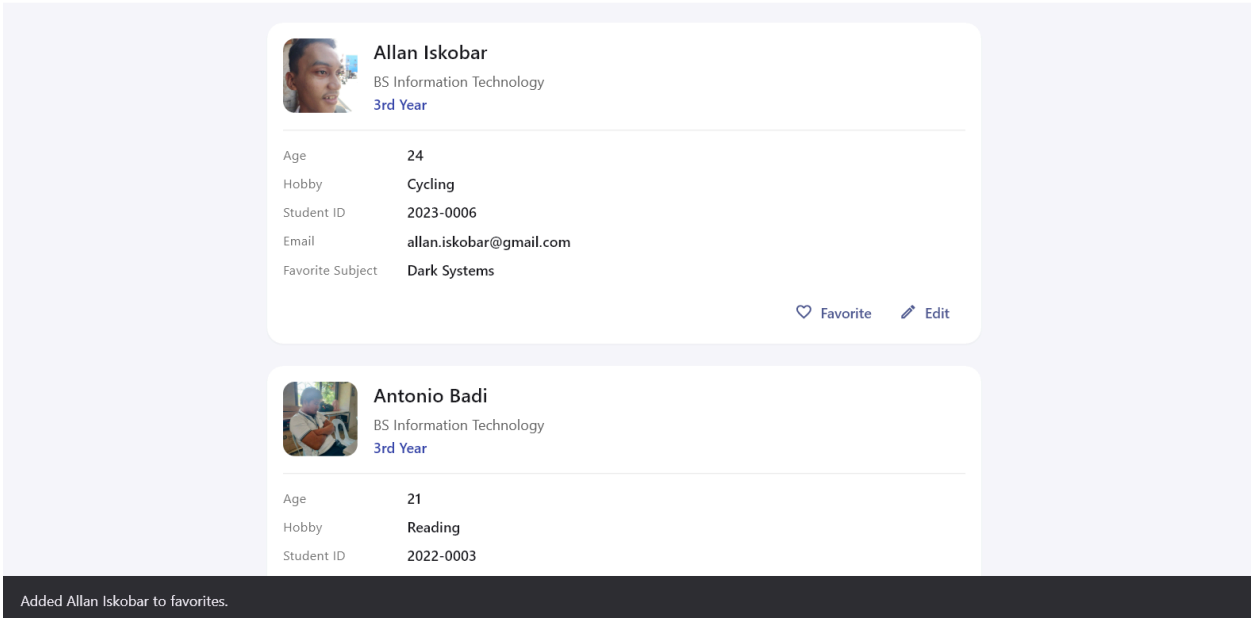


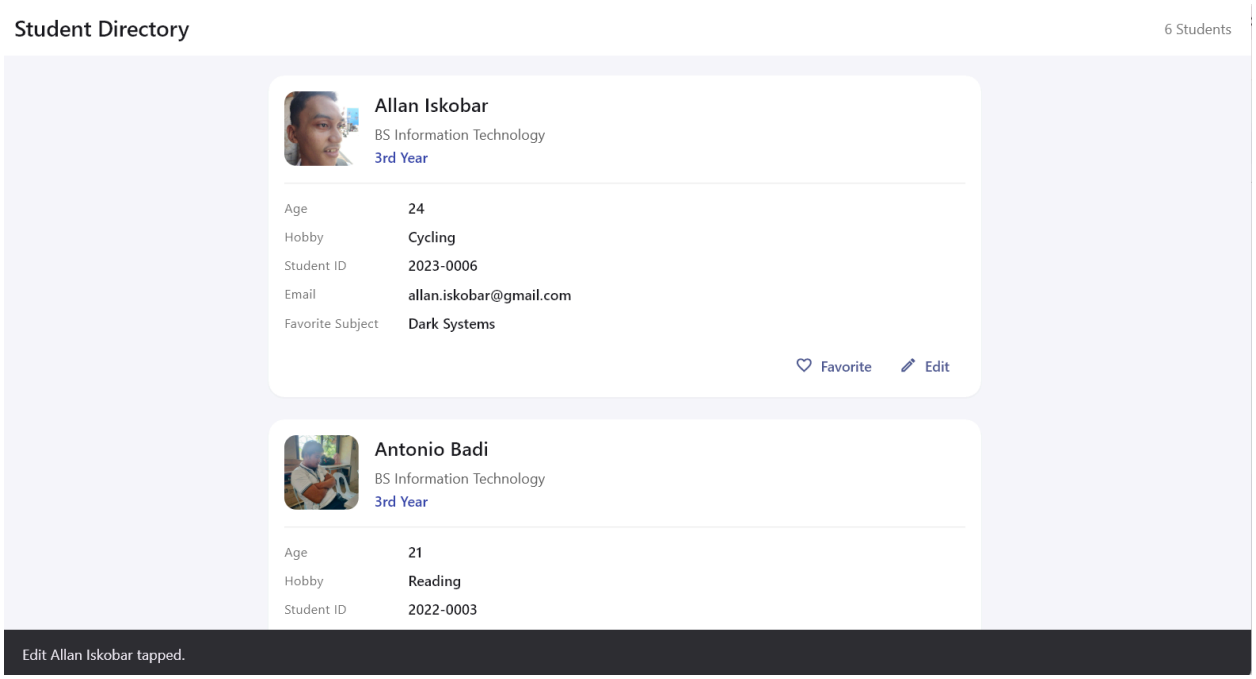
- `onPressed` inside your code





- Button reacting or terminal message





🏴 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

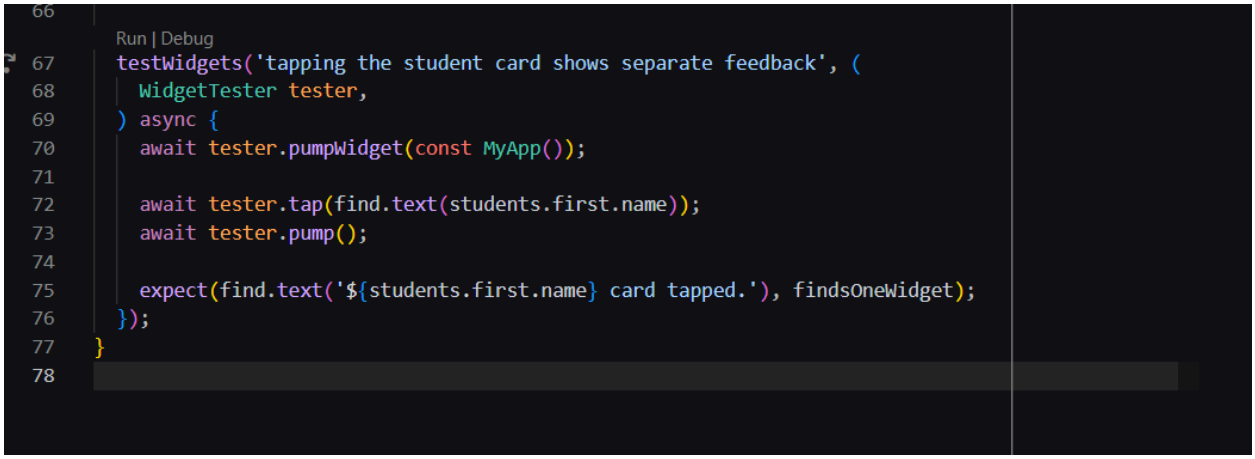
The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

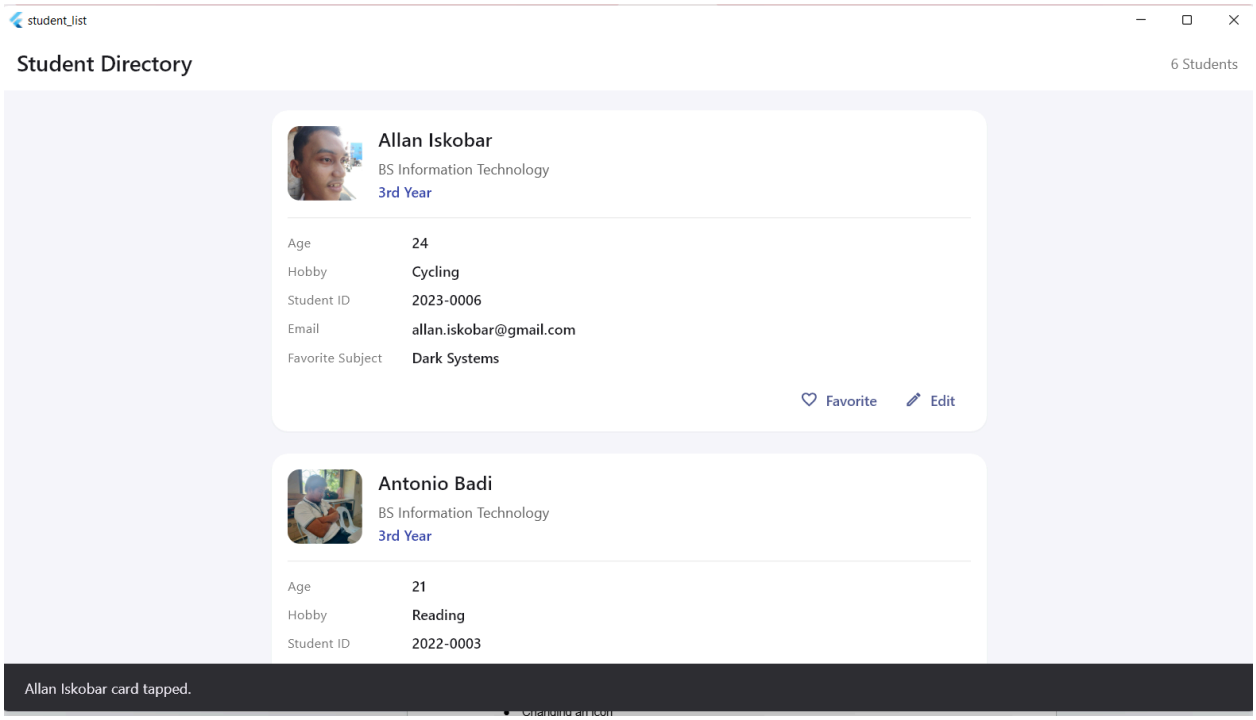
Screenshot:

- `onTap` in your code



```
66
    Run | Debug
67  testWidgets('tapping the student card shows separate feedback', (
68    WidgetTester tester,
69  ) async {
70    await tester.pumpWidget(const MyApp());
71
72    await tester.tap(find.text(students.first.name));
73    await tester.pump();
74
75    expect(find.text('${students.first.name} card tapped.'), findsOneWidget);
76  });
77  }
78
```

- Student Card being tapped

- 

🏴 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

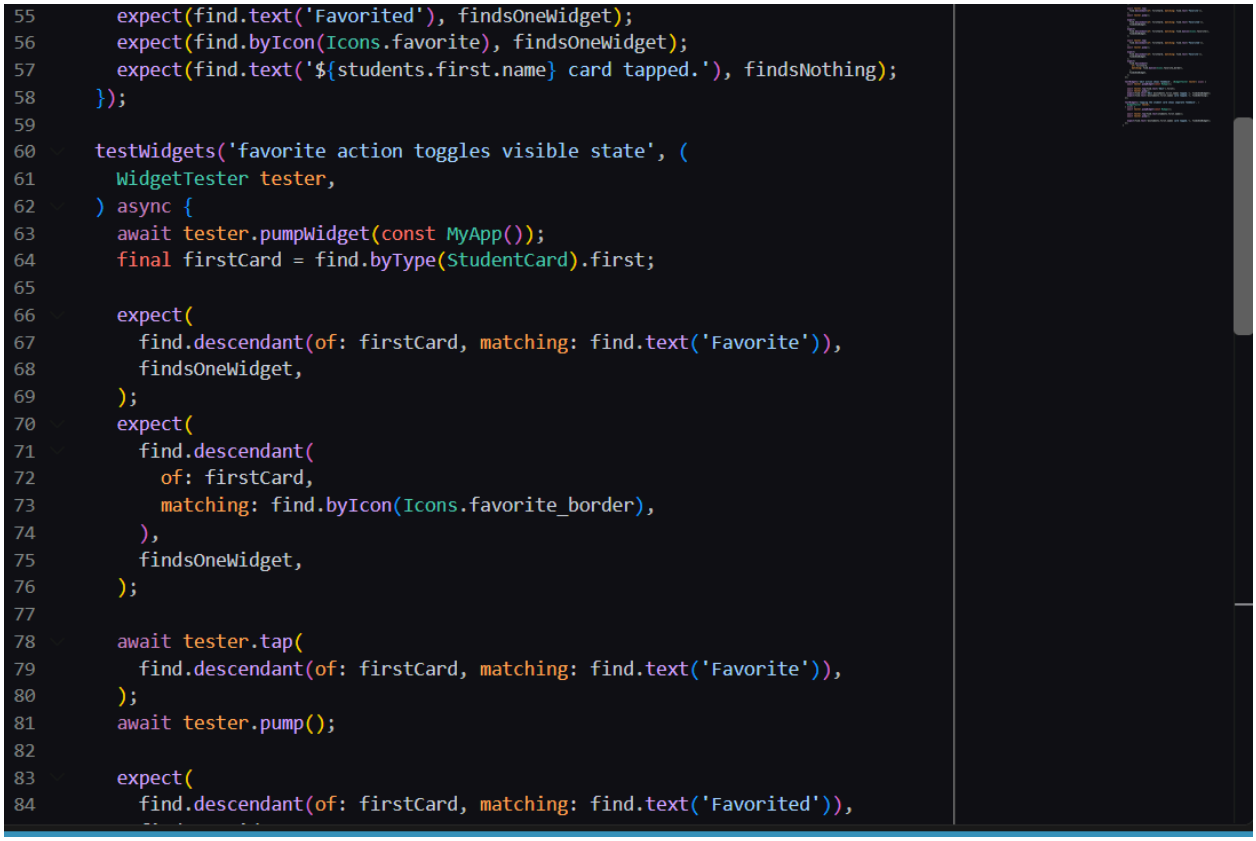
The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

The screen should visibly change immediately after pressing the button.

Screenshot:

- `setState` in your code

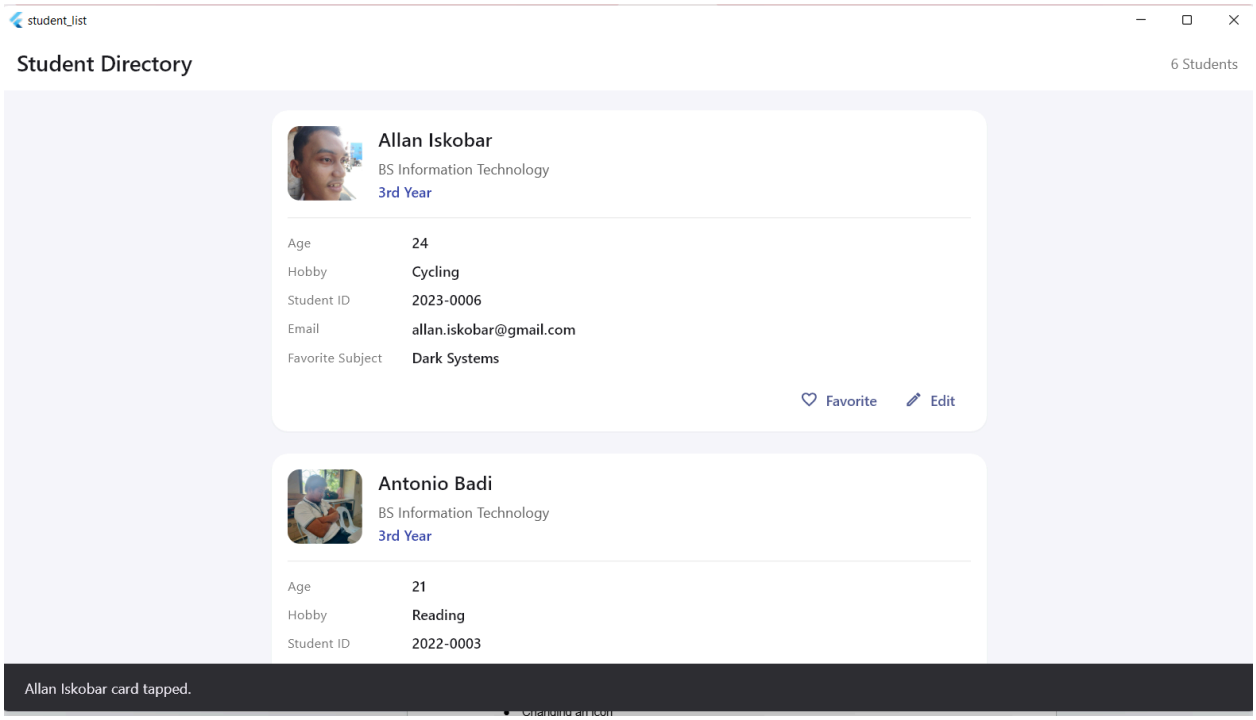


```
55      expect(find.text('Favorited'), findsOneWidget);
56      expect(find.byIcon(Icons.favorite), findsOneWidget);
57      expect(find.text('${students.first.name} card tapped.'), findsNothing);
58    });
59
60    testWidgets('favorite action toggles visible state', (
61      WidgetTester tester,
62    ) async {
63      await tester.pumpWidget(const MyApp());
64      final firstCard = find.byType(StudentCard).first;
65
66      expect(
67        find.descendant(of: firstCard, matching: find.text('Favorite')),
68        findsOneWidget,
69      );
70      expect(
71        find.descendant(
72          of: firstCard,
73          matching: find.byIcon(Icons.favorite_border),
74        ),
75        findsOneWidget,
76      );
77
78      await tester.tap(
79        find.descendant(of: firstCard, matching: find.text('Favorite')),
80      );
81      await tester.pump();
82
83      expect(
84        find.descendant(of: firstCard, matching: find.text('Favorited')),
```

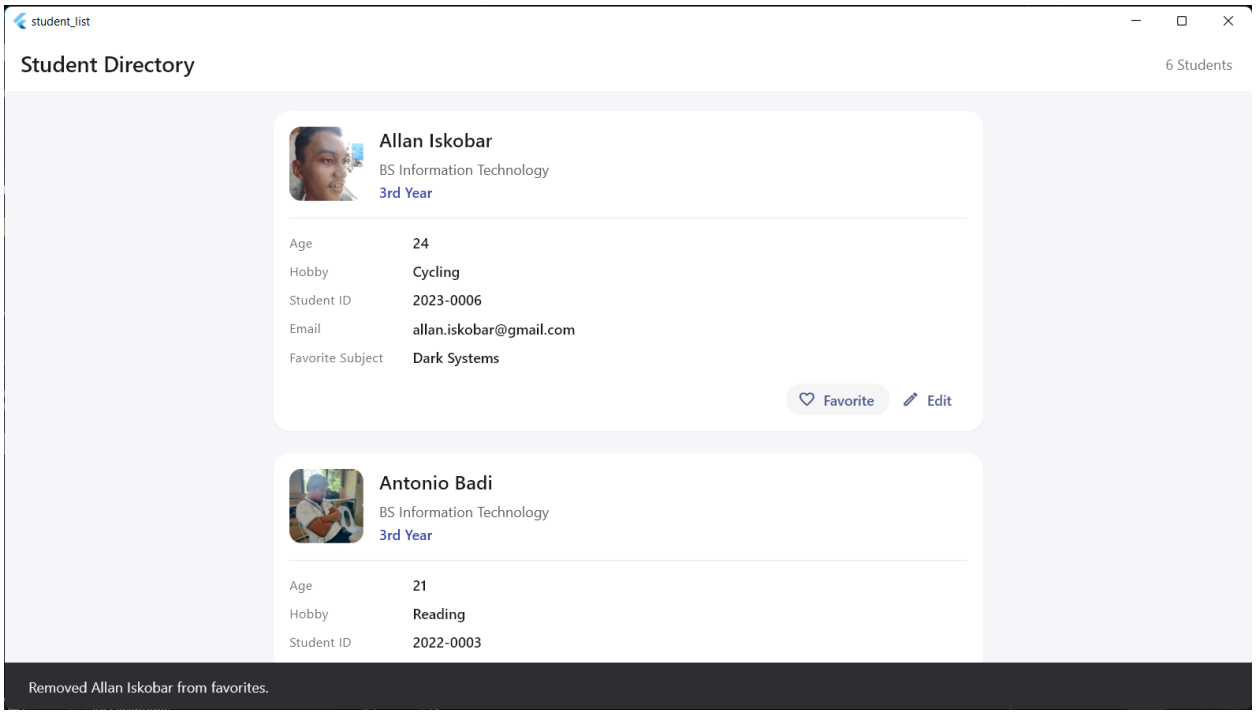
```
78        await tester.tap(
79          find.descendant(of: firstCard, matching: find.text('Favorite')),
80        );
81        await tester.pump();
82
83        expect(
84          find.descendant(of: firstCard, matching: find.text('Favorited')),
85          findsOneWidget,
86        );
87        expect(
88          find.descendant(of: firstCard, matching: find.byIcon(Icons.favorite)),
89          findsOneWidget,
90        );
91
92        await tester.tap(
93          find.descendant(of: firstCard, matching: find.text('Favorited')),
94        );
95        await tester.pump();
96
97        expect(
98          find.descendant(of: firstCard, matching: find.text('Favorite')),
99          findsOneWidget,
100       );
101       expect(
102         find.descendant(
103           of: firstCard,
104           matching: find.byIcon(Icons.favorite_border),
105         ),
106         findsOneWidget,
107       );
108     });
109
110   testWidgets('edit action shows feedback', (WidgetTester tester) async {
```

- Before and after interaction

  Before:

- 

After:



🏴 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

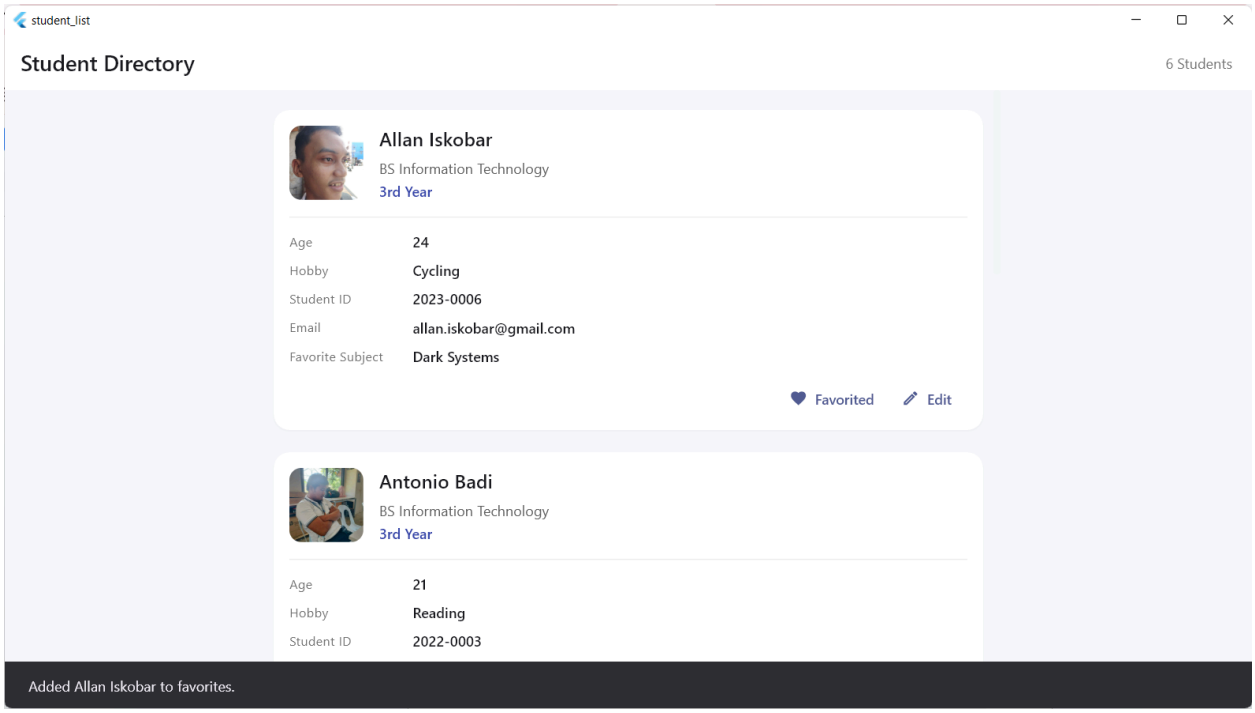
Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

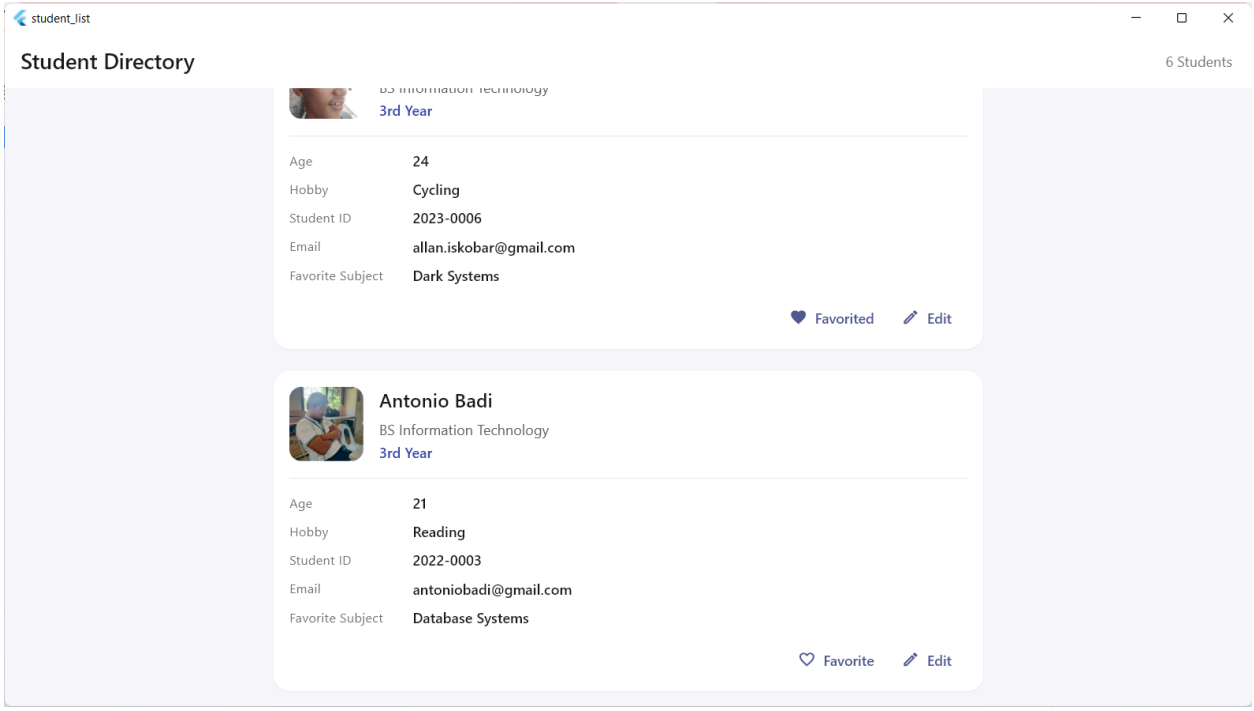
Both should keep their own state.

Screenshot:

- Favorited student



- Non-favorited student



- Favorite icon changing



🏴 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart `List`.

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B

- Student C

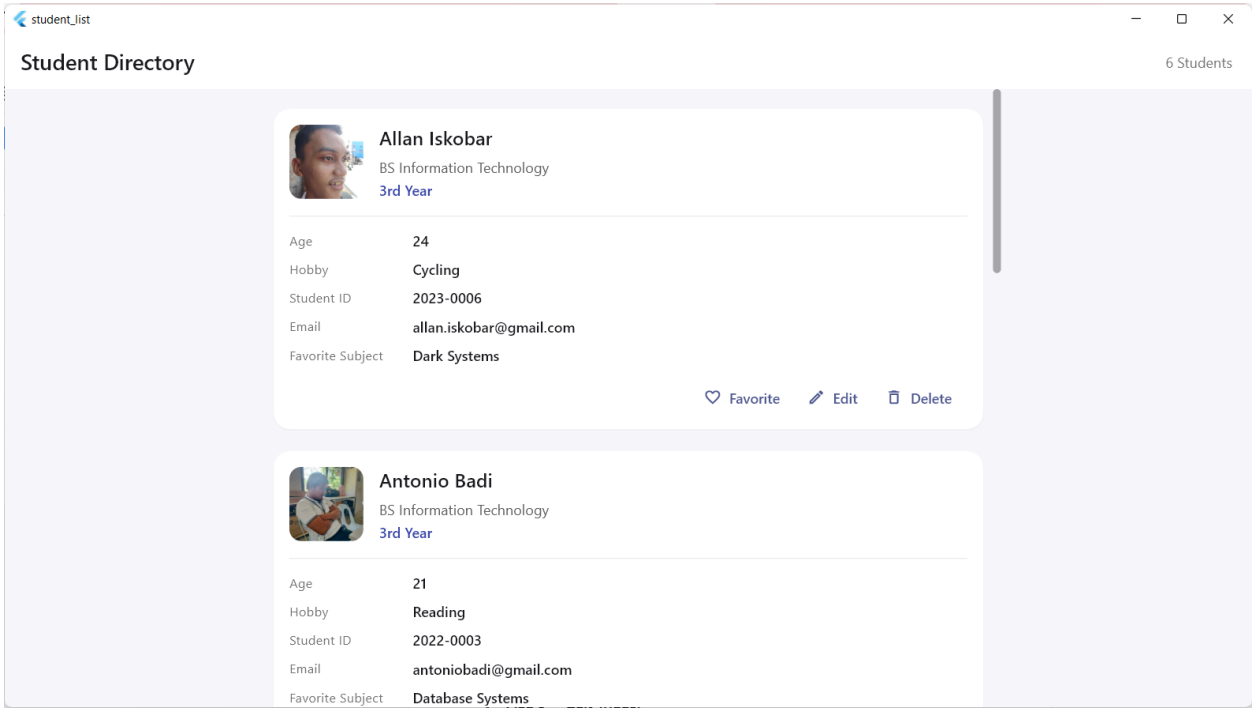
After deleting Student B:

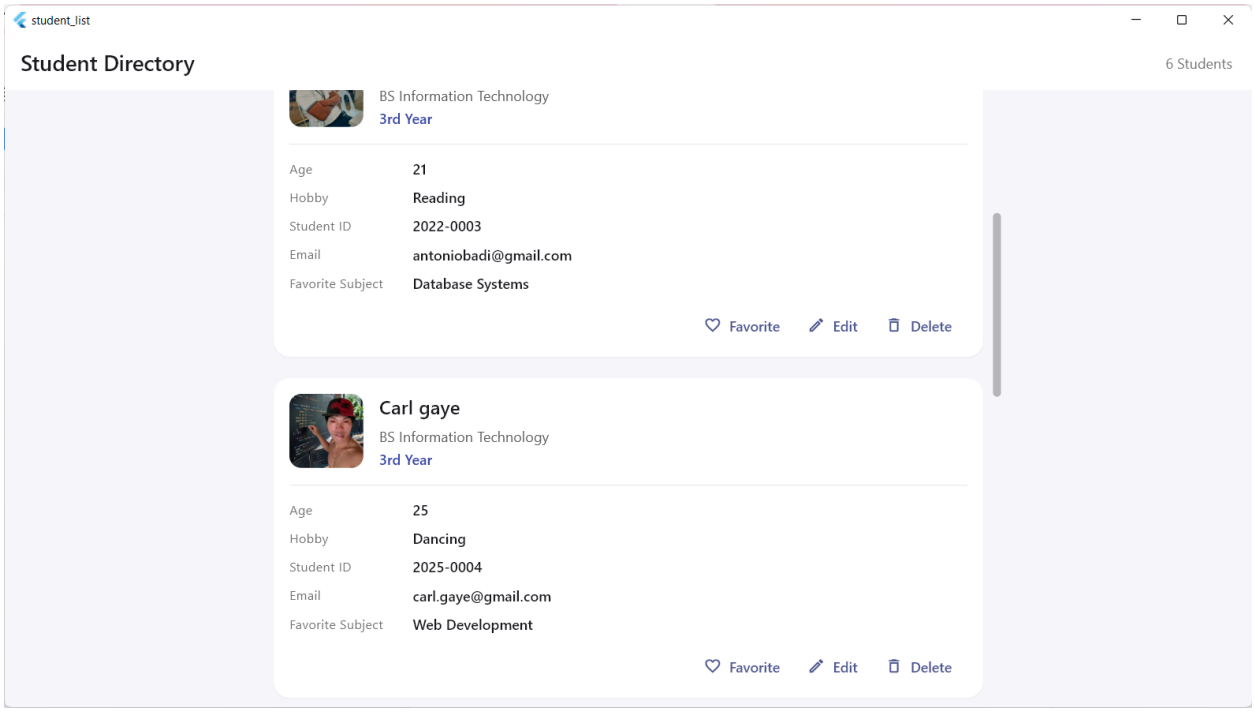
- Student A
- Student C

No empty spaces should remain.

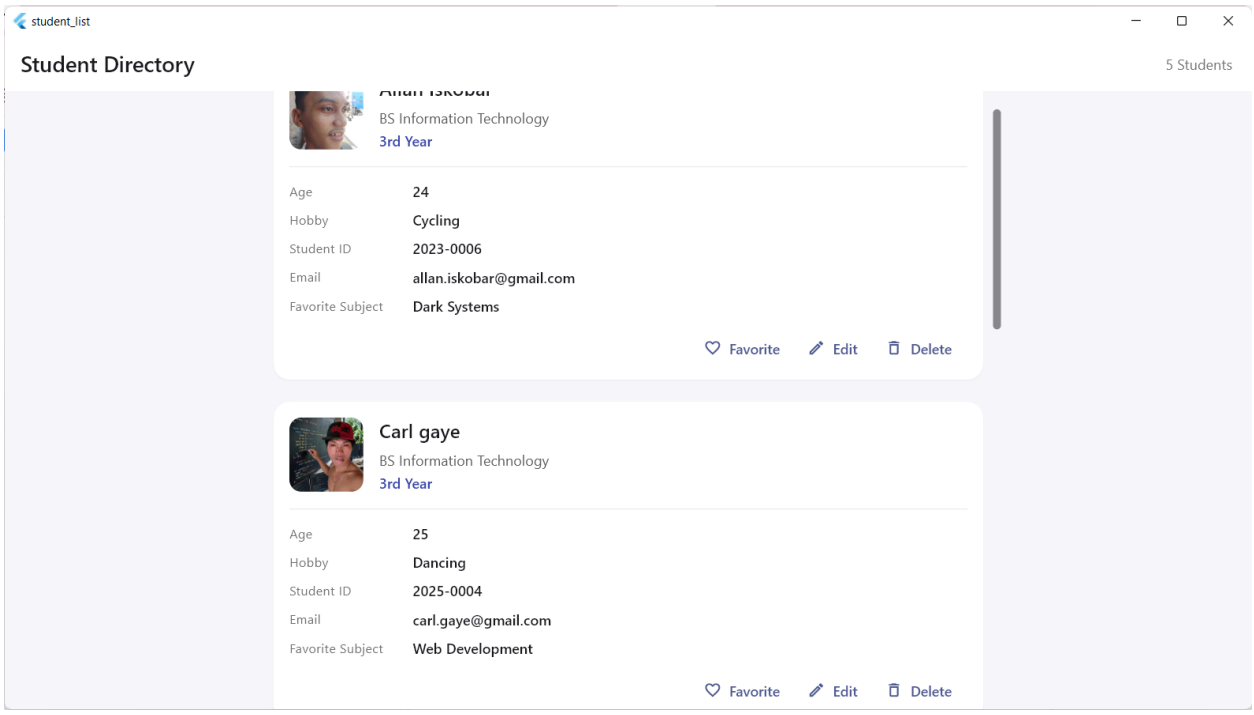
Screenshot:

- Student before deletion





- Student after deletion



🏴 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

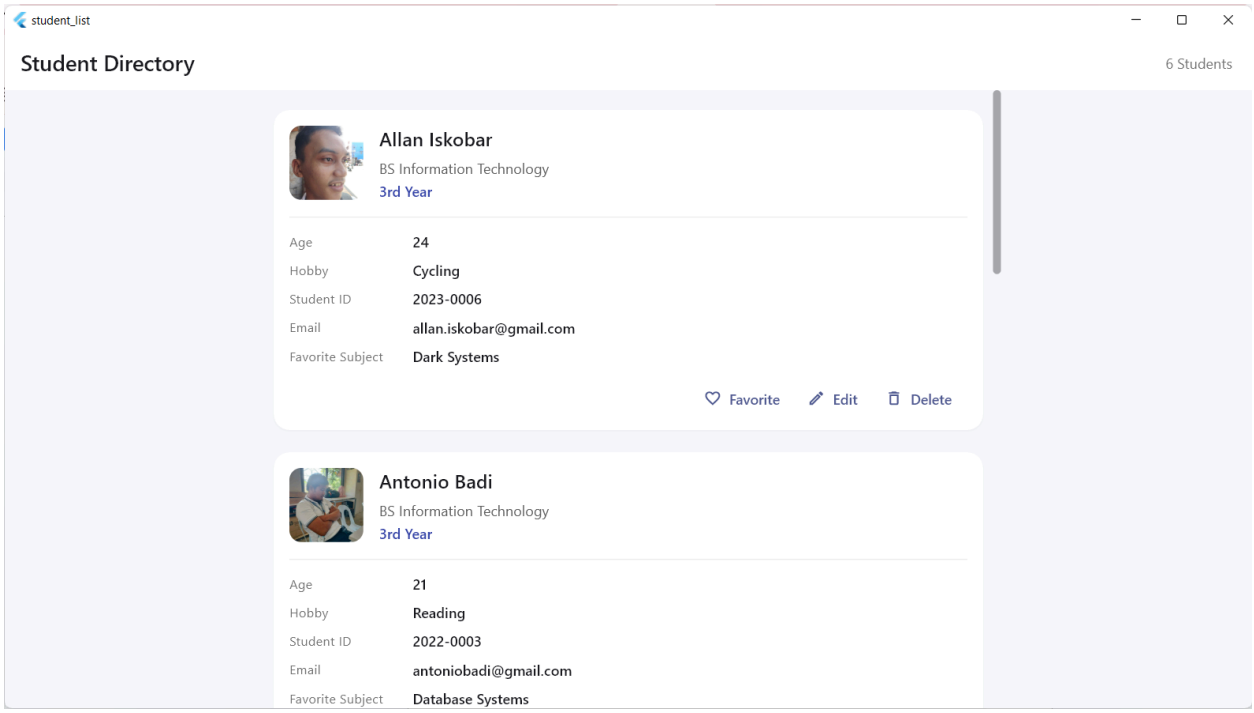
The goal is simply to launch an edit action, not build the complete editor yet.

Test

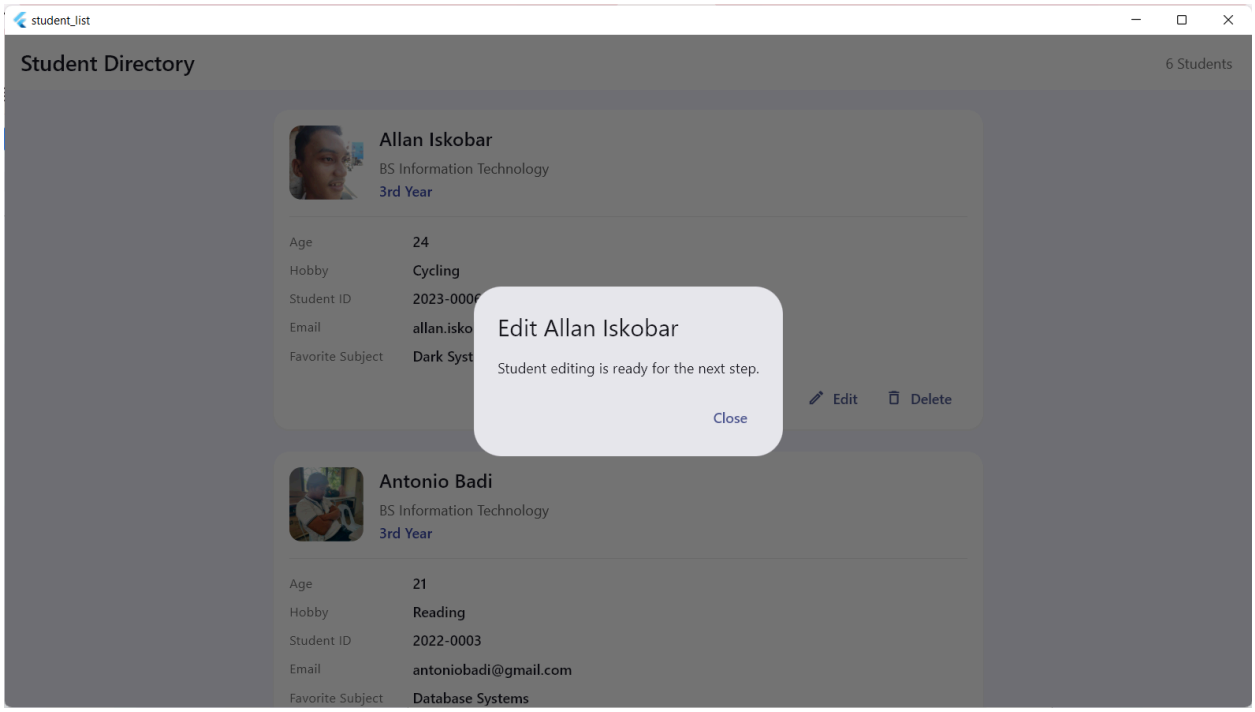
Pressing Edit should open your chosen interface.

Screenshot:

- Edit button



- Opened dialog, snackbar, or bottom sheet



🏴 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

Your application should continue functioning correctly without affecting unrelated students.

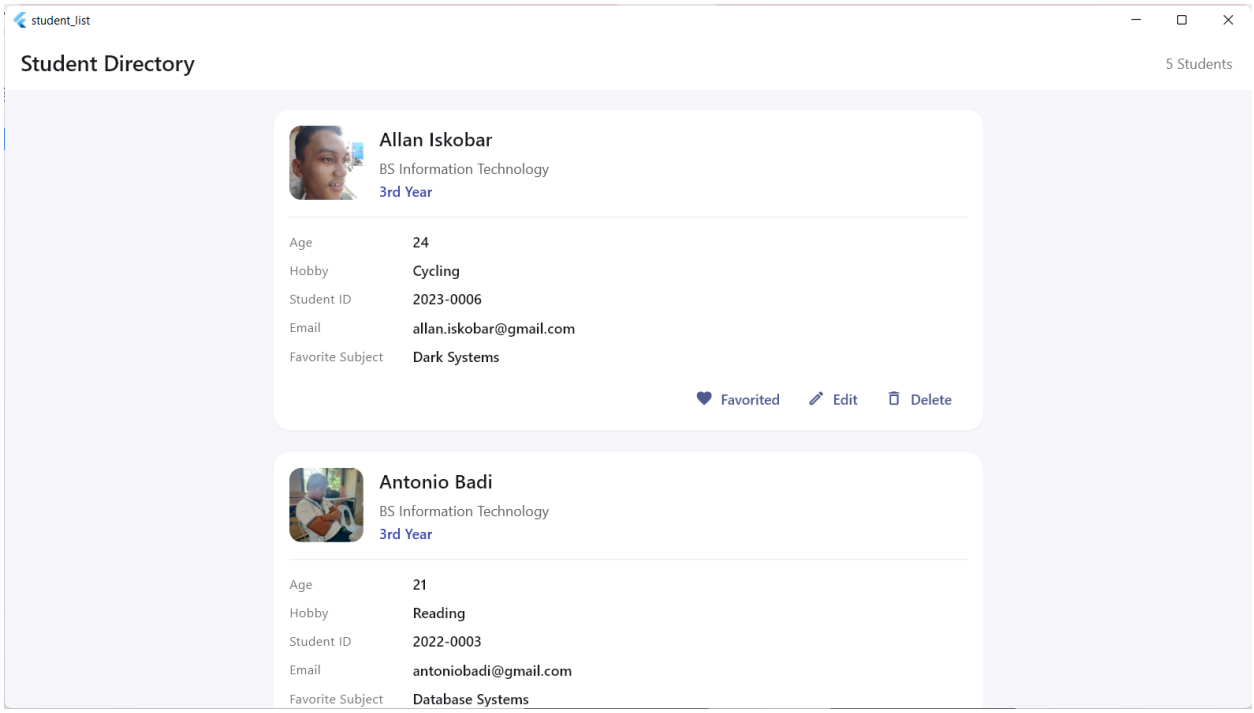
Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

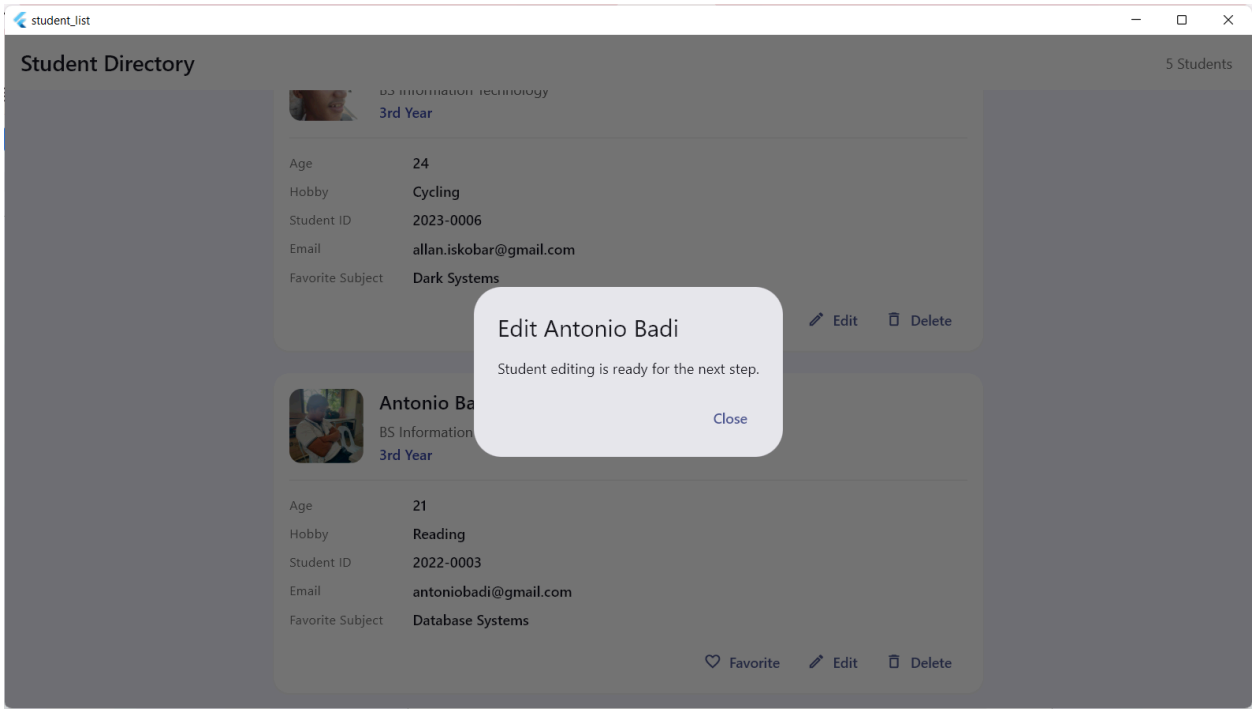
Screenshot:

- Multiple interactions working together

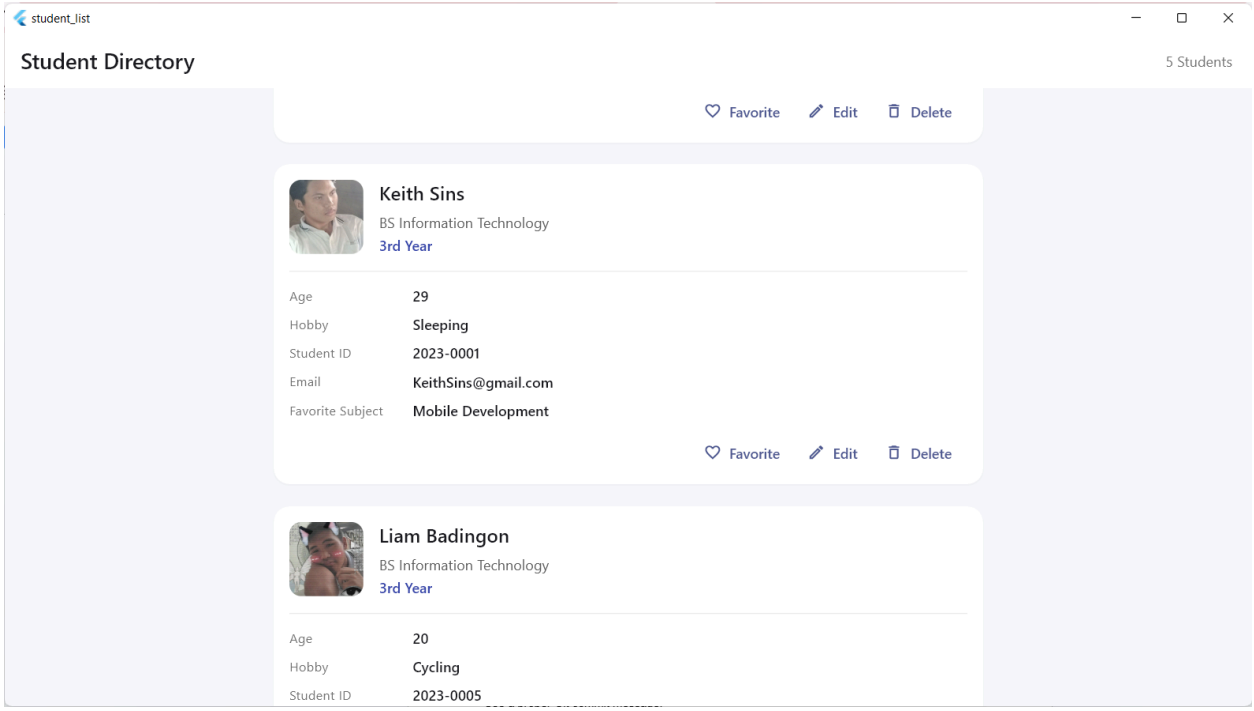
  Favorite Student A :



  Edit Student B:



Deleted Student C:



- Application still functioning normally

🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit